

Reflexions especulars

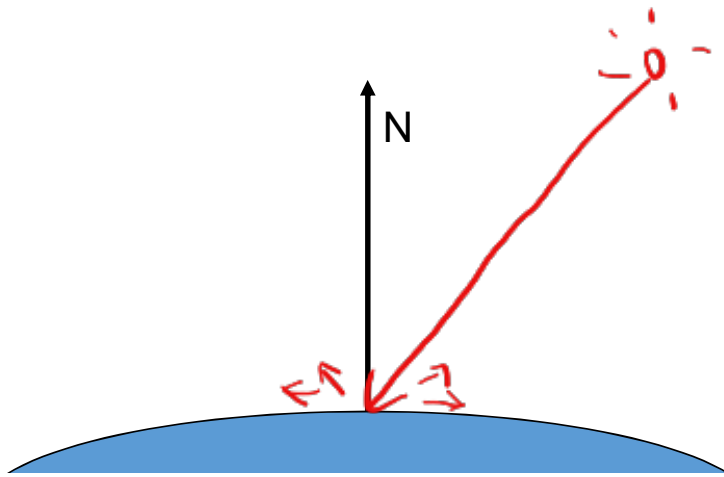
Carlos Andújar

Nov 2022

Introducció

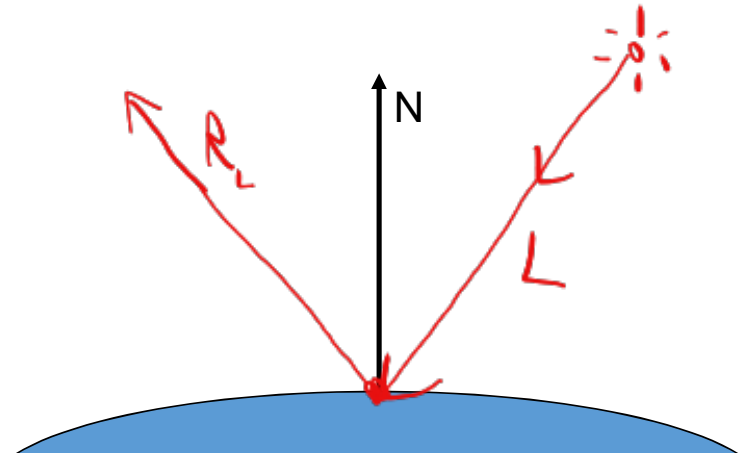


Reflexió difosa



En tota igual d'on ve
de la llum

Reflexió especular



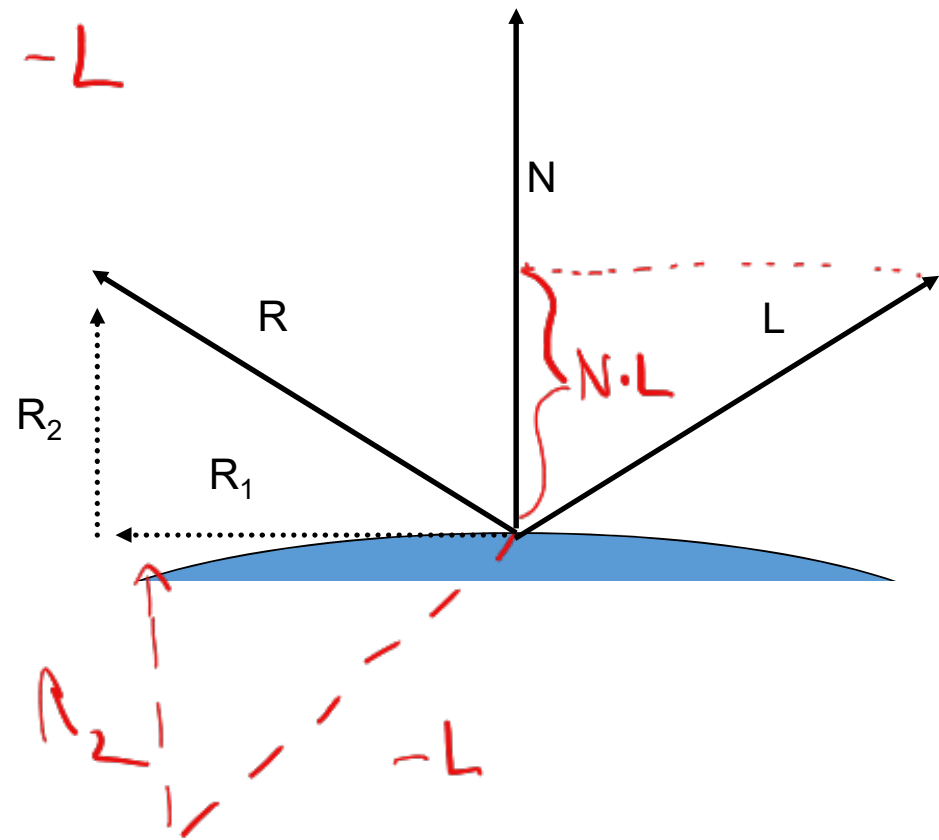
No tota igual d'on
ve la llum

Vector reflectit

$$R = R_2 + R_1 \equiv 2(N \cdot L)N - L$$

$$R_2 = (N \cdot L)N$$

$$R_1 = -L + R_2$$



Vector reflectit

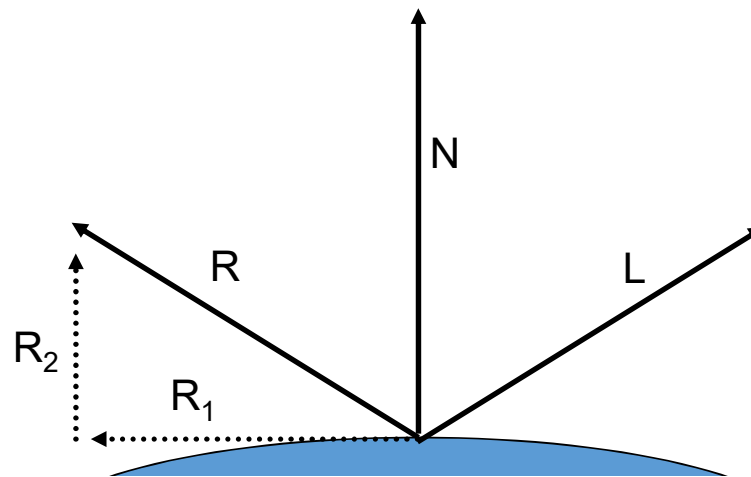
$$R = R_1 + R_2$$

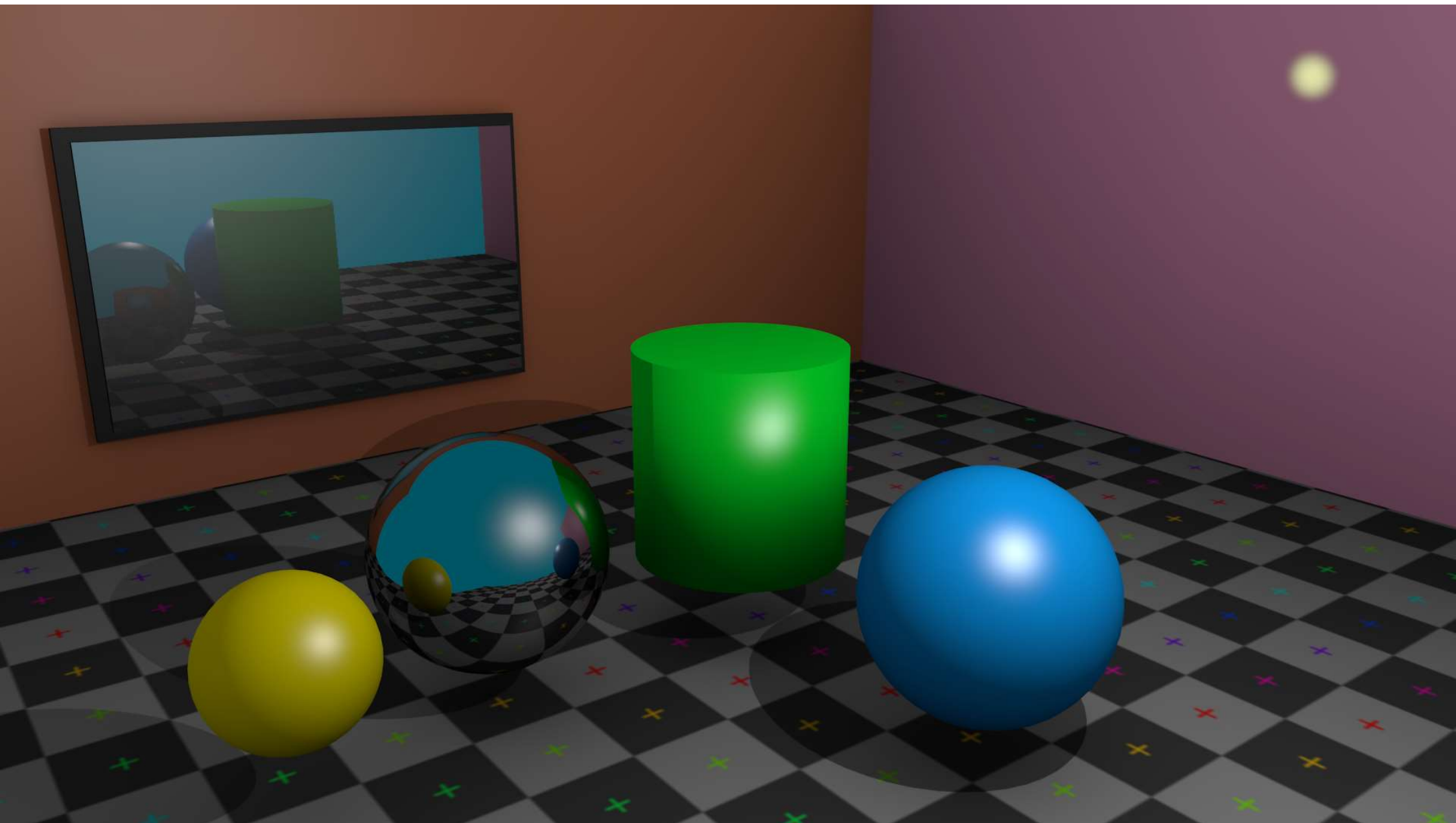
$$R_1 = -L + R_2$$

$$R = 2R_2 - L$$

$$R_2 = (N \cdot L)N$$

$$\mathbf{R} = 2 (\mathbf{N} \cdot \mathbf{L}) \mathbf{N} - \mathbf{L}$$





Mètodes

- Ray-tracing
- Reflexions basada en **objectes virtuals**

- Modelats
- Reflectits (sense stencil test)
- Reflectits (amb stencil test)
- Textures dinàmiques

} Pre: mirall pla
Reflexió exacta

- **Environment mapping**

- Sphere mapping
- Cube mapping

} 1st amb mirall esfèric
No tan exacta.

Reflexions amb objectes virtuals

Mètodes

- Ray-tracing
- Reflexions basada en **objectes virtuals**
 - Modelats
 - Reflectits (sense stencil test)
 - Reflectits (amb stencil test)
 - Textures dinàmiques
- **Environment mapping**
 - Sphere mapping
 - Cube mapping

Objectes virtuals



obj

mirall

pos. virtual
el dir 2 veg
el obj!

Mètodes

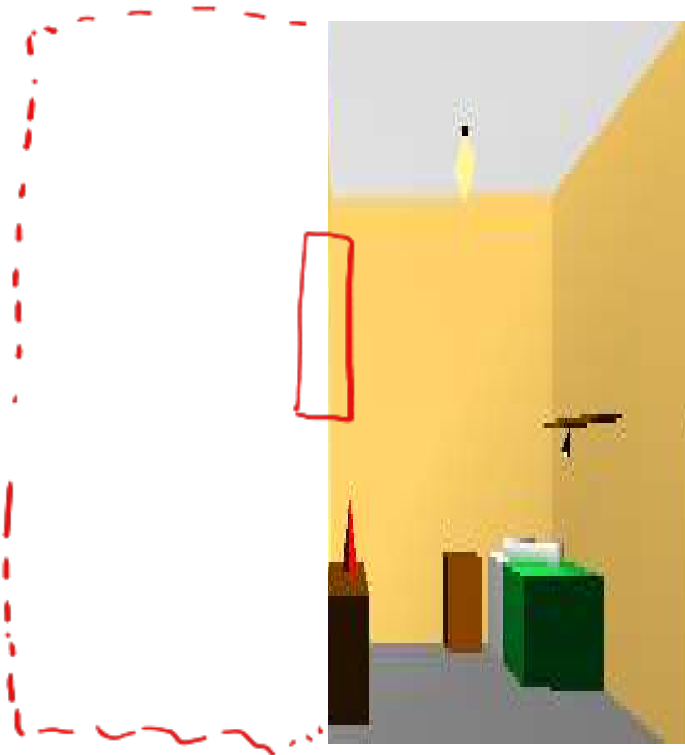
- Ray-tracing
- Reflexions basada en **objectes virtuals**
 - Modelats
 - Reflectits (sense stencil test)
 - Reflectits (amb stencil test)
 - Textures dinàmiques
- **Environment mapping**
 - Sphere mapping
 - Cube mapping

Mandelbrot, duplicar!

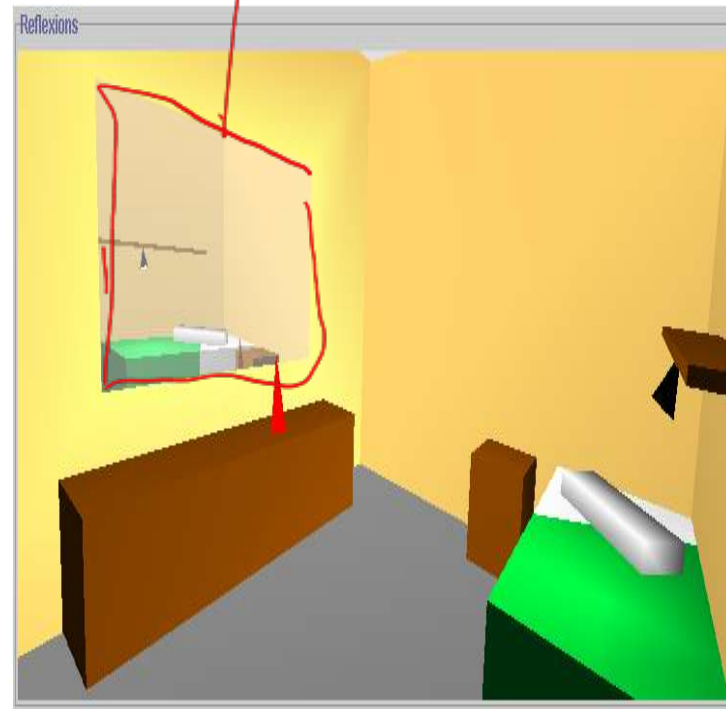
Objectes virtuals

X
↳ Si es man obj?

✓
↳ execucio i
re optimitza
re a duplici
tats



Sarat! Duplicar execucio!



Mètodes

- Ray-tracing
- Reflexions basada en **objectes virtuals**
 - Modelats
 - Reflectits (sense stencil test)
 - Reflectits (amb stencil test)
 - Textures dinàmiques
- **Environment mapping**
 - Sphere mapping
 - Cube mapping

} → reflexió amb matriu de VS

Reflexió Matriu simetria = $S(-1, 1, 1) = \begin{bmatrix} -1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{bmatrix}$

Algorisme (versió 1)

// 1. Dibuixar els objectes en posició virtual (amb M_R)

```
glPushMatrix();
glMultMatrix(matriu_simetria)
glLightfv(GL_LIGHT0, GL_POSITION, pos);
glCullFace(GL_FRONT);
dibuixar(escena);
glPopMatrix();
```

← eliminar la "front" pos
en veritat són la back!
(Signe det)

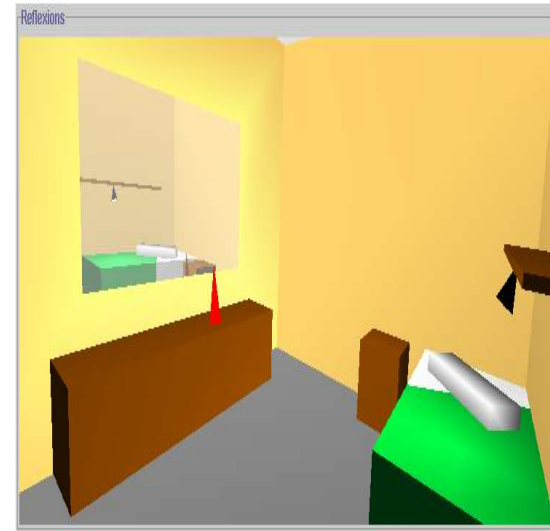
// 2. Dibuixar el mirall semi-transparent

```
glLightfv(GL_LIGHT0, GL_POSITION, pos);
glCullFace(GL_BACK);
dibuixar(mirall);
```

← la arrefem

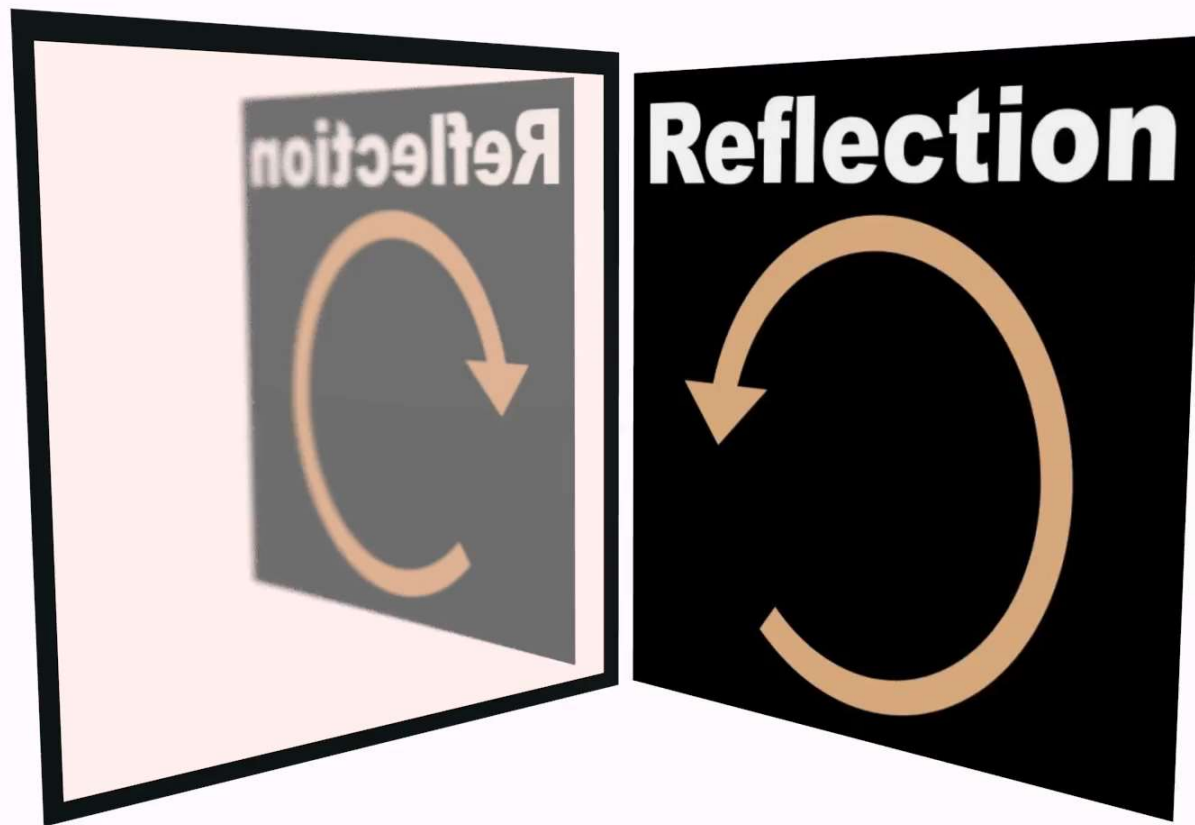
// 3. Dibuixar els objectes en posició real (Sense M_R)

```
dibuixar(escena);
```



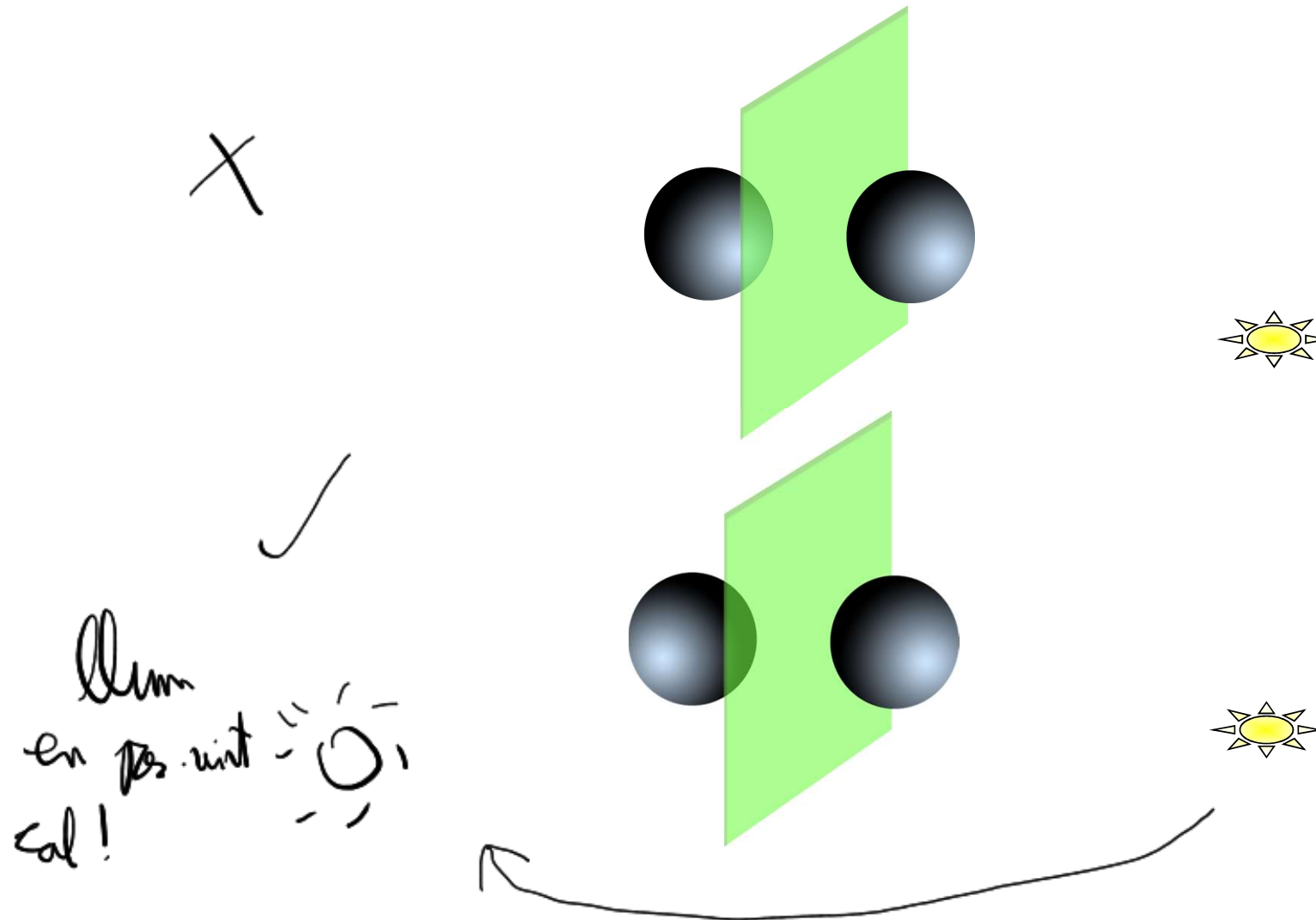
Front Face / Back Face

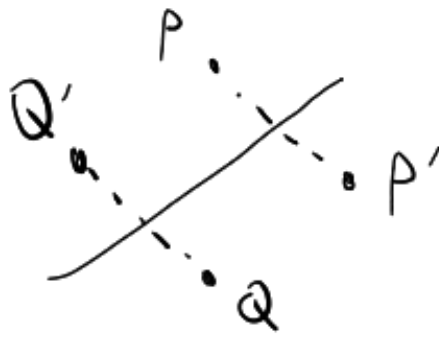
Inversió ordre



La llum no es reflecta!

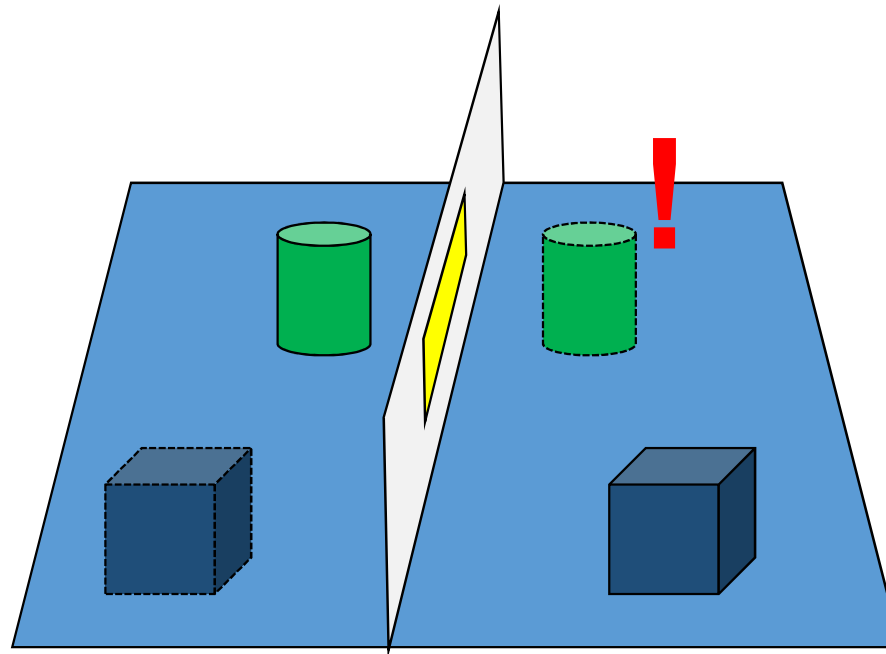
Reflexió de les fonts de llum





Limitacions

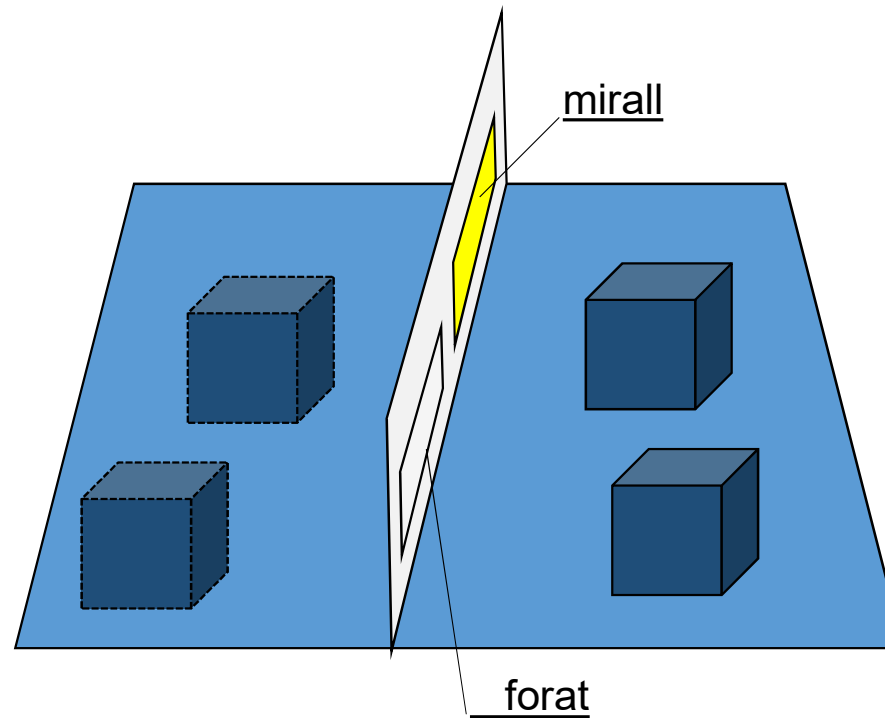
Assumeix que els objectes virtuals estan en el semiespai positiu del pla del mirall.



Limitacions

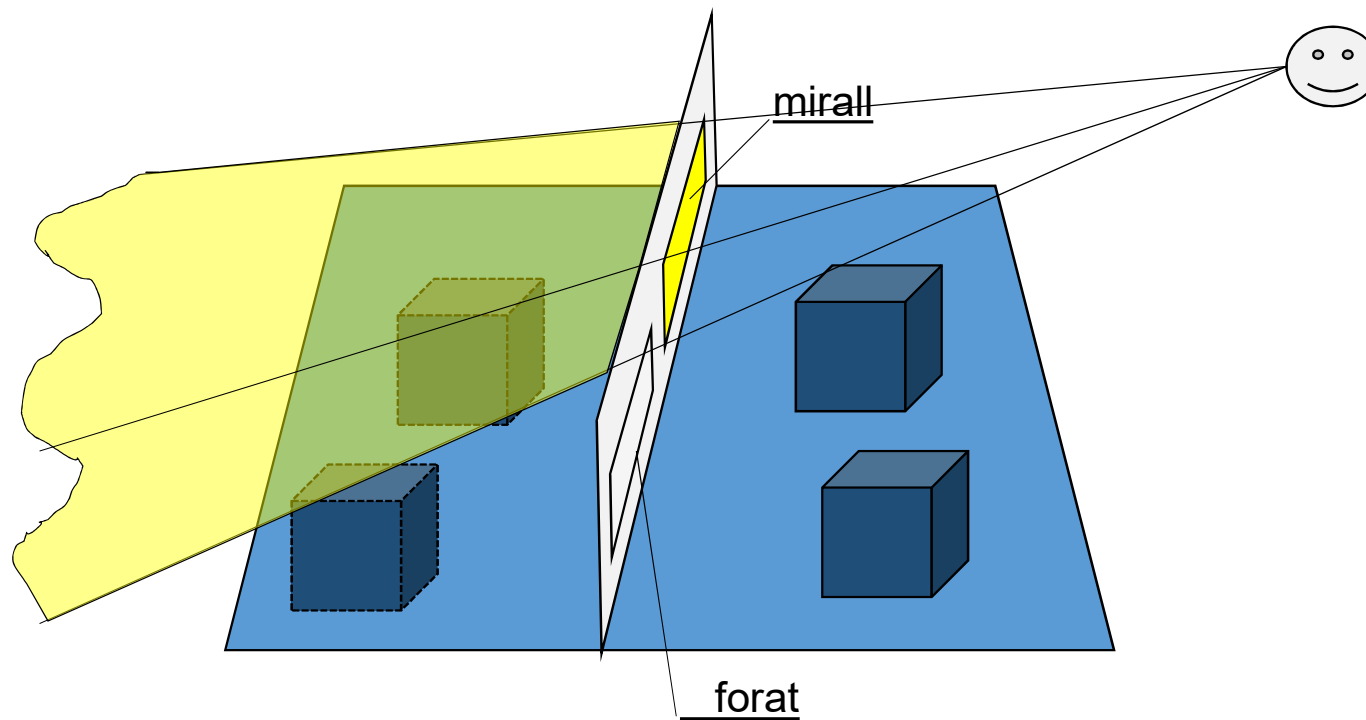
Assumeix que els objectes virtuals només es veuran a través del forat del mirall.

*No hi ha entre
mirall i forat*



Solució 1 *complicat*

Dibuixar els objectes virtuals amb plans de retallat
addicionals – `glClipPlane()`



Solució 2 *stencil més fàcil !!*

Usar stencil per limitar els objectes virtuals a la regió ocupada pel mirall.

Mètodes

- Ray-tracing
- Reflexions basada en **objectes virtuals**

- Modelats
- Reflectits (sense stencil test)
- Reflectits (amb stencil test)
- Textures dinàmiques

← limitar els reflectits a un camp de visió

- **Environment mapping**
 - Sphere mapping
 - Cube mapping

Stencil

Test
True

Algorisme (versió 2)

// 1. Dibuixem el mirall a l'stencil buffer

Op (1)
Stencil
Test True
replace per 1
z - ton
Si no es veu el de darrere
`glEnable(GL_STENCIL_TEST);`
`glStencilFunc(GL_ALWAYS, 1, 1);`
`glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);`
`glDepthMask(GL_FALSE); glColorMask(GL_FALSE...);`
`dibuixar(mirall);`

// 2. Dibuixem els objectes virtuals

X *== 1?*
`glDepthMask(GL_TRUE); glColorMask(GL_TRUE...);`
`glStencilFunc(GL_EQUAL, 1, 1);`
`glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);`
`glPushMatrix(); glMultMatrix(matriu simetria)`
`glLightfv(GL_LIGHT0, GL_POSITION, pos);`
`glCullFace(GL_FRONT);`
`dibuixar(escena);`
`glPopMatrix();`

// 3. Dibuixem el mirall semitransparent

X
X
`glDisable(GL_STENCIL_TEST);`
`glLightfv(GL_LIGHT0, GL_POSITION, pos);`
`glCullFace(GL_BACK);`
`dibuixar(mirall);`

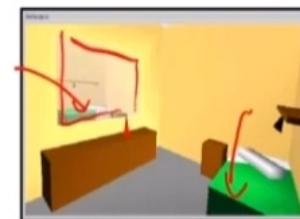
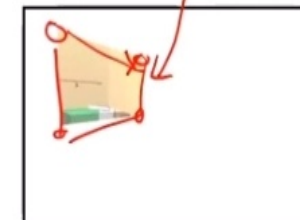
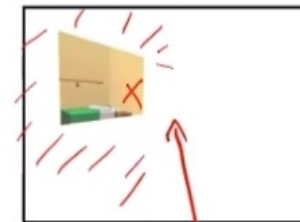
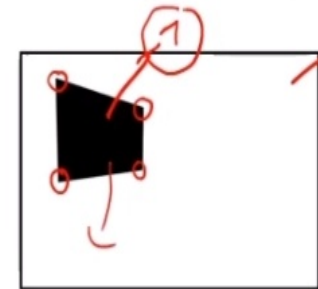
// 4. Dibuixem els objectes reals

`dibuixar(escena);`

Color buffer

Stencil Buffer

Test Z



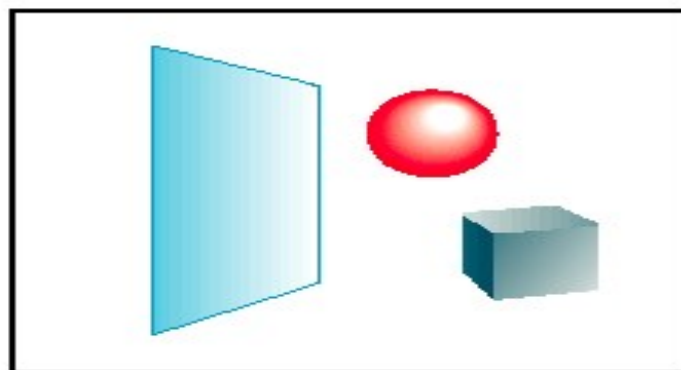
Mètodes

- Ray-tracing
- Reflexions basada en **objectes virtuals**
 - Modelats
 - Reflectits (sense stencil test)
 - Reflectits (amb stencil test)
 - Textures dinàmiques
- **Environment mapping**
 - Sphere mapping
 - Cube mapping

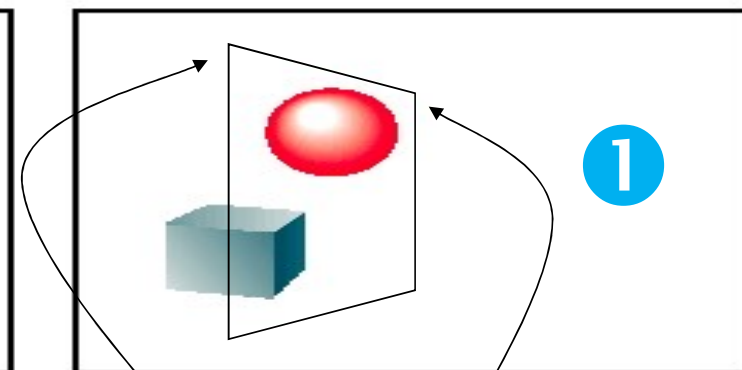
Textures Dinàmiques

Vent: es poden calcular recalcular tot

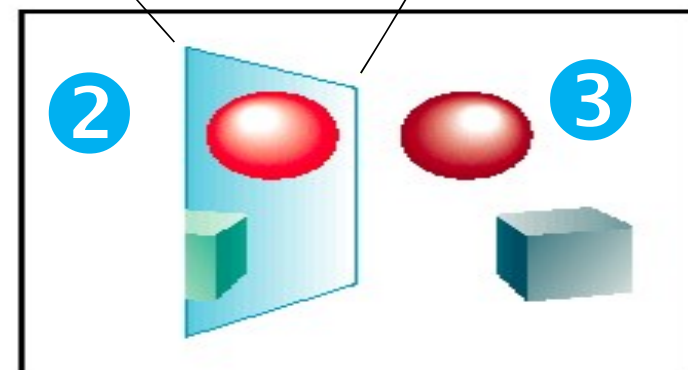
Textures dinàmiques



Escena original



Escena reflectida



Escena final

① Dibuixar en por. virtual

② Crear text amb info de tots ls objectes

③ Dibuixar quad del mirall, amb text

④ Dibuixar en por real

FS

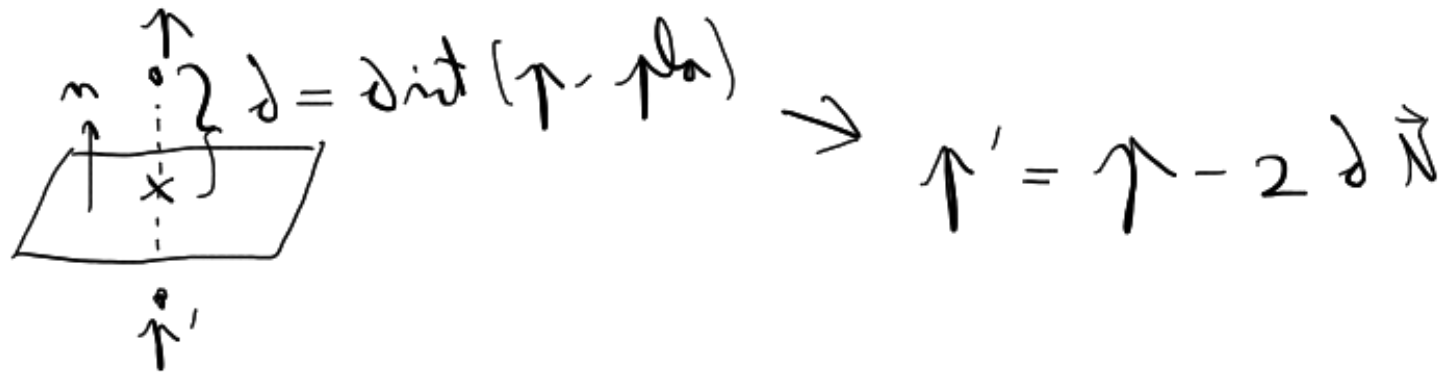
$$\text{fragColor} = \text{texture}(\text{colorMap}, \frac{\text{gl-FragCoord.xy}}{\text{viewport}})$$

[0,1]

26
viewport $\rightarrow 1024^2$
texture $\rightarrow 1024^2$

$$\text{pla } x=0 \Rightarrow M_R = S(-1, 1, 1)$$

Matriu de reflexió $(a, b, c, d) \rightarrow ax + by + cz + d = 0$



$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \uparrow x \\ \uparrow y \\ \uparrow z \\ 1 \end{bmatrix} = \begin{bmatrix} \uparrow x' \\ \uparrow y' \\ \uparrow z' \\ 1 \end{bmatrix}$$

Matriu de reflexió

Matriu de reflexió respecte un pla (a,b,c,d):

$$\begin{bmatrix} 1-2a^2 & -2ba & -2ca & -2da \\ -2ba & 1-2b^2 & -2cb & -2db \\ -2ca & -2cb & 1-2c^2 & -2dc \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

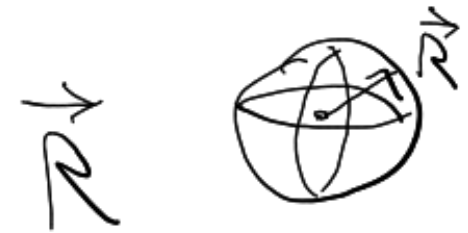
Environment mapping

Supporta reflexion en whatever objects!

Environment map

Donat una direcció arbitrària R , ens retorna el color de l'entorn en direcció R

$\text{color} = \text{environmentMap}(R)$

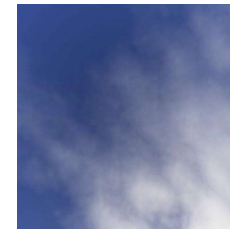


Representació com a textura

Proj equidistangular



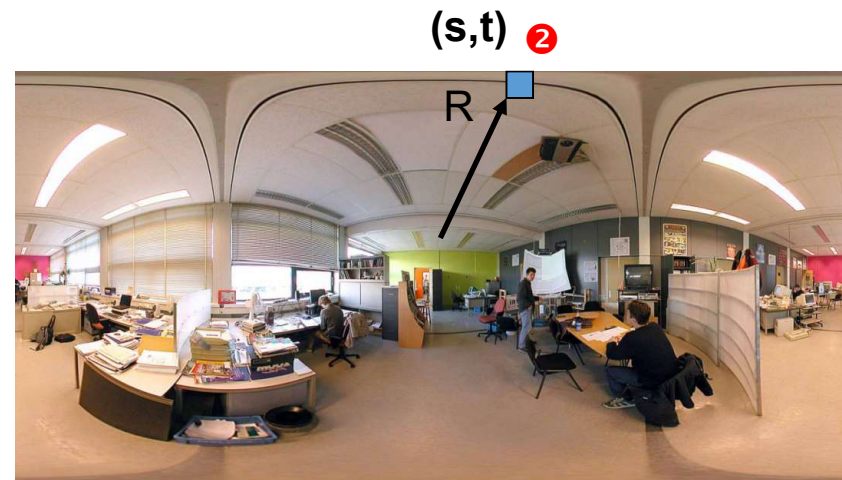
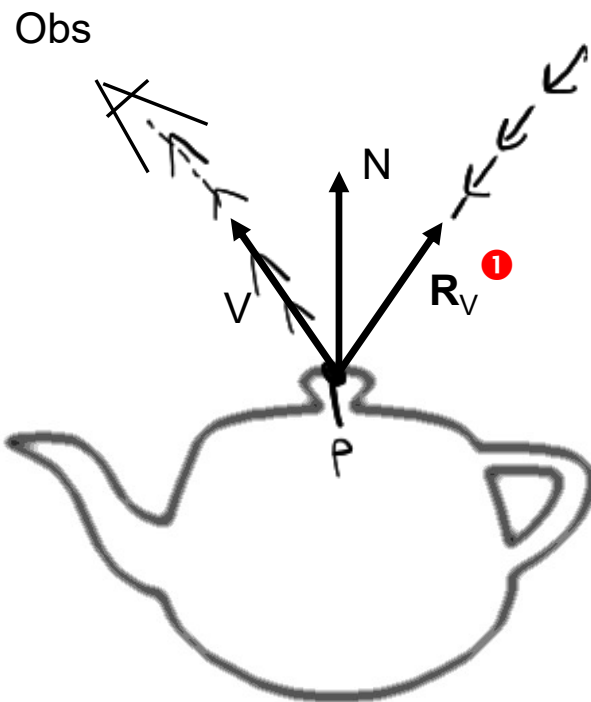
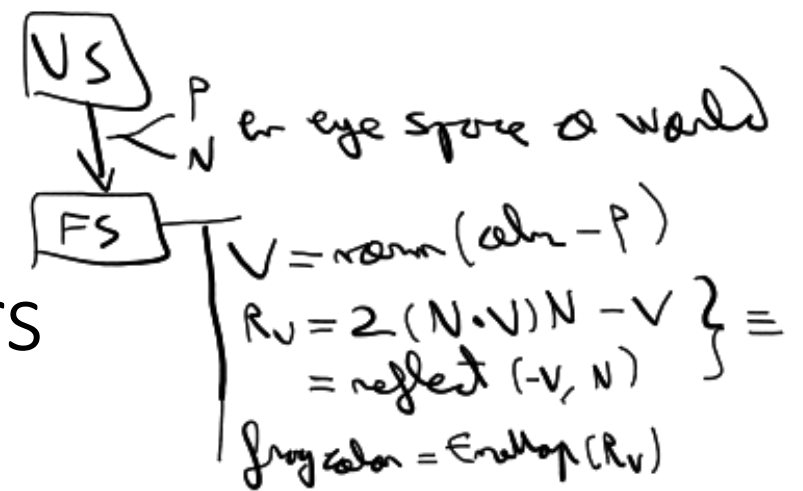
Cube Map



Sphere Map

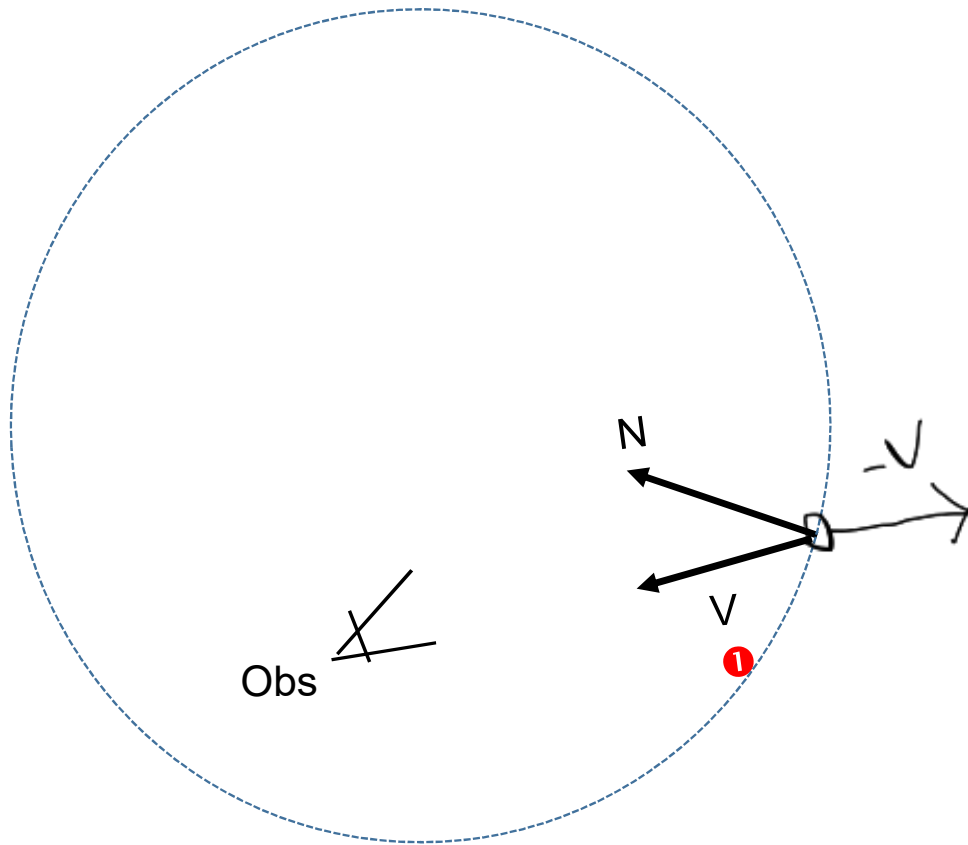
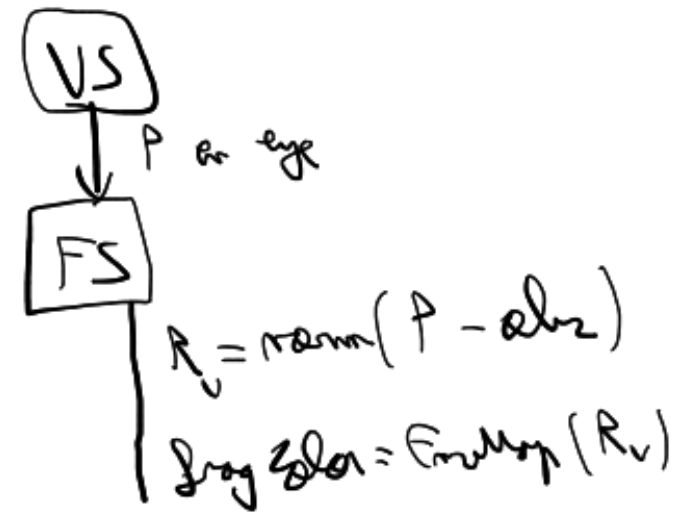


Ús per reflexions especulars



→ real for

Ús com a entorn (background)



(s,t) 2



Sphere mapping

Sphere map

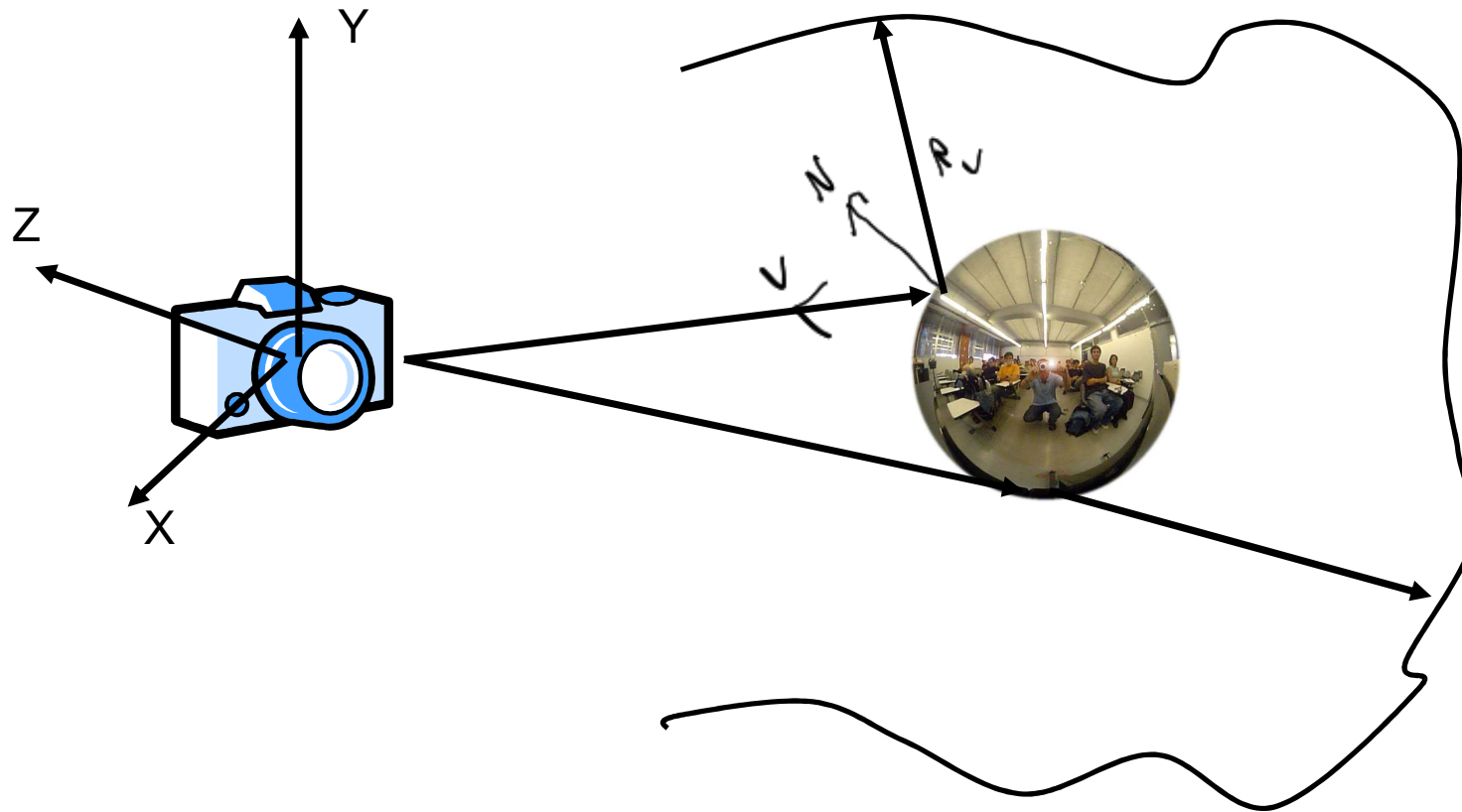


Sphere map

Hand with Reflecting Sphere by **M. C. Escher**.
Lithograph, 1935.
Official M.C. Escher website.

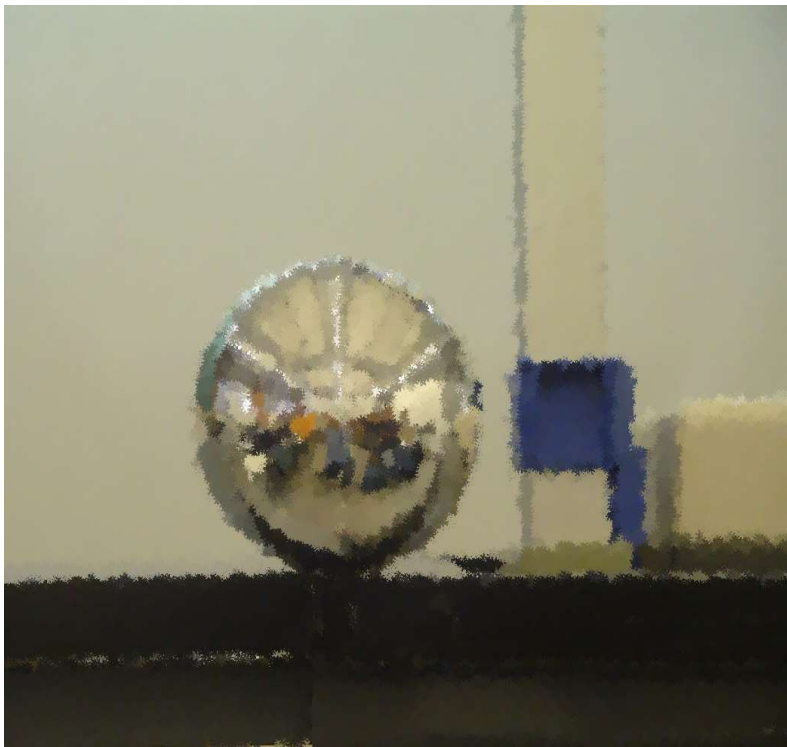


Construcció de sphere maps



Exemple sphere map

Sense retallar

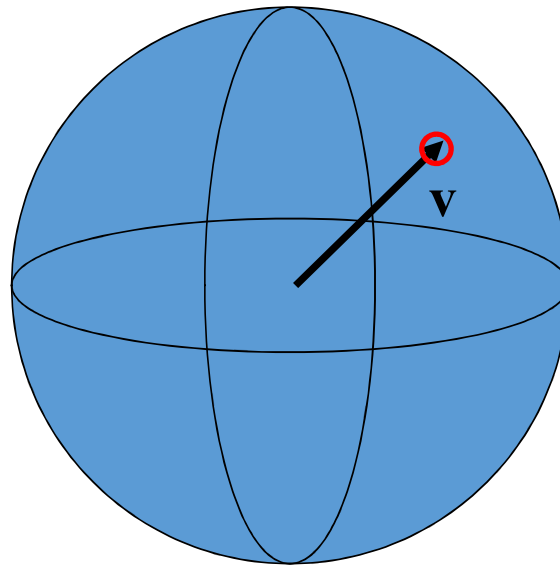


Retallat



Propietats

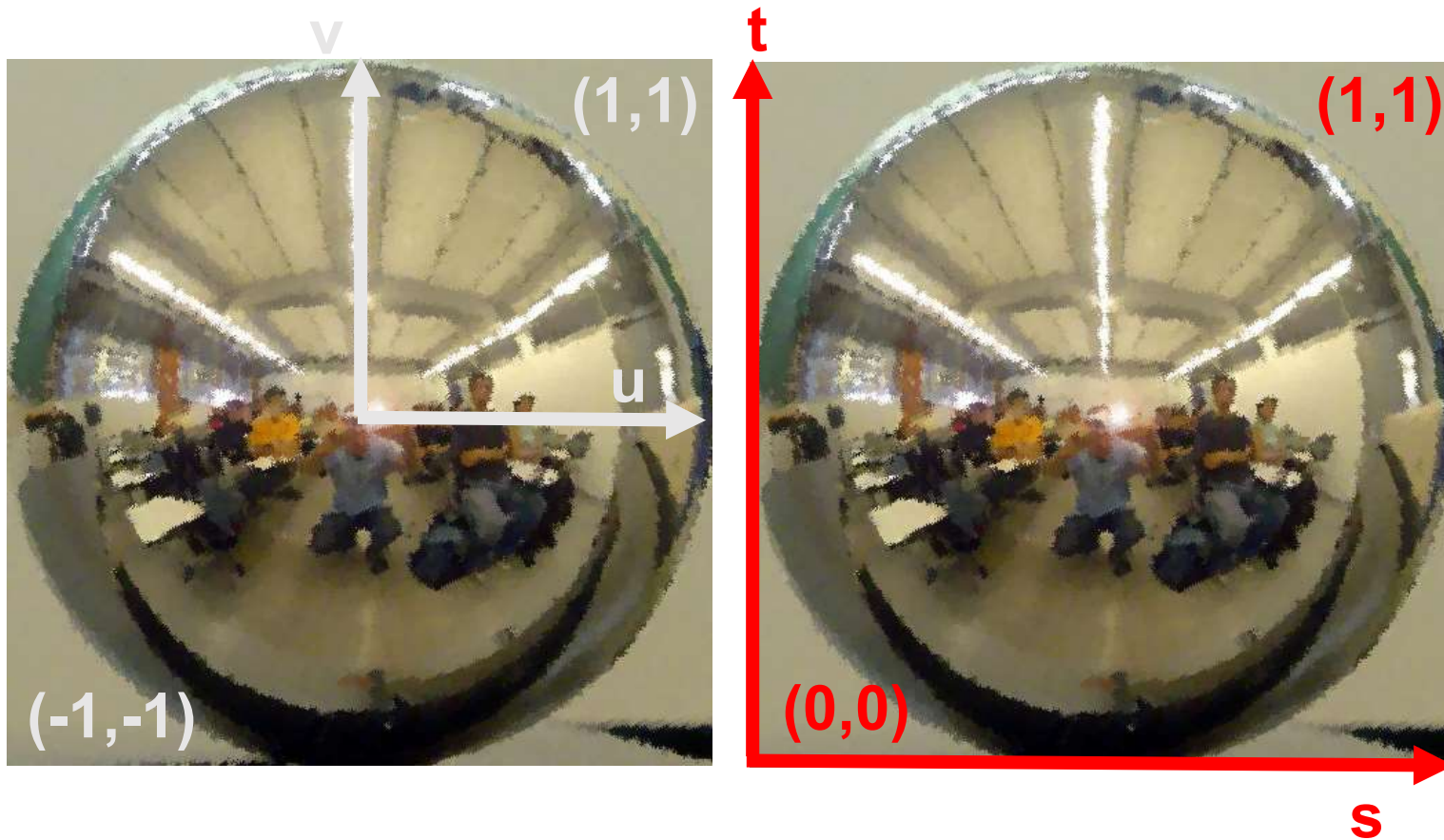
- De la textura només se n'aprofita el cercle inscrit
- Conté informació de aproximadament tot l'entorn (totes direccions)
- Distorsió considerable a prop de la vora del cercle



Exemple

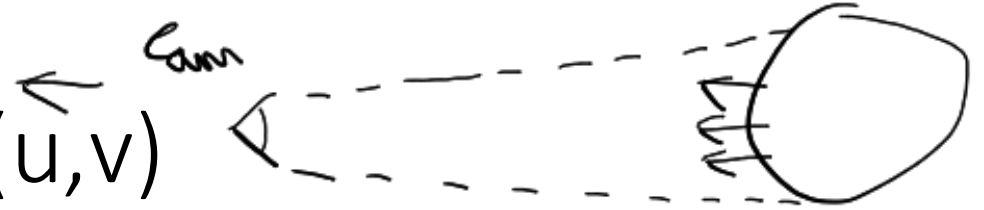


Coordenades $(u,v) \leftrightarrow (s,t)$

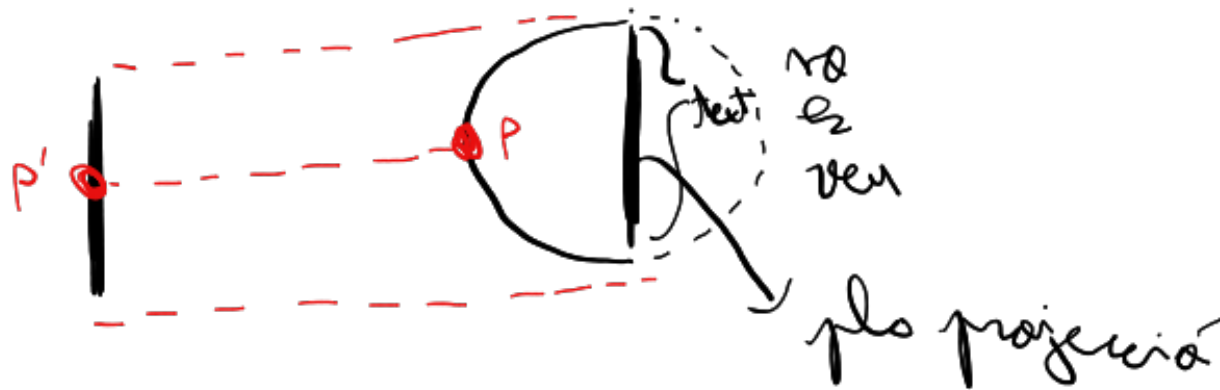


oerometrisch = om al infinit + lang $\Rightarrow$ + detail

Relació entre vector R i $(\vec{u}, v)$



Superan alienanthicus!



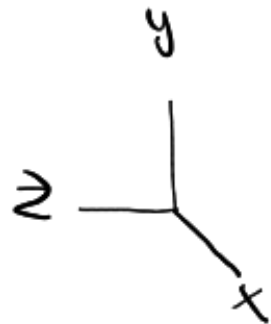
Relació entre vector R i (u,v)

$$R = (2n_z n_x, 2n_z n_y, 2n_z^2 - 1)$$

$$\uparrow_z = n_z = \sqrt{\frac{n_z + 1}{2}}$$

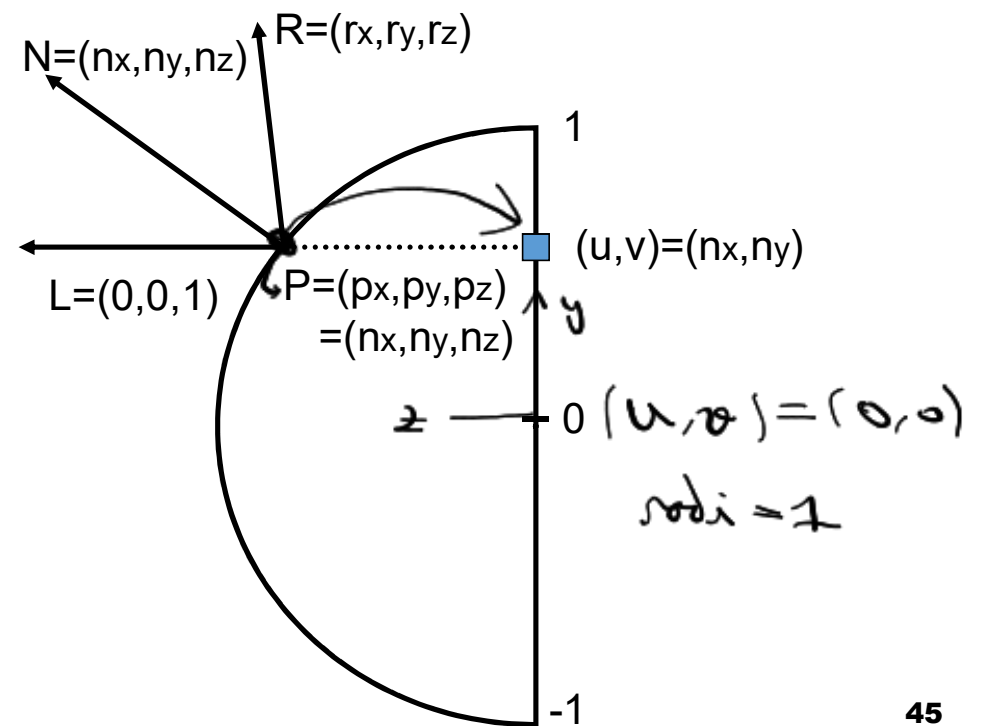
$$\uparrow_x = n_x = \frac{n_x}{2n_z} \rightarrow u$$

$$\uparrow_y = n_y = \frac{n_y}{2n_z} \rightarrow v$$



$$s = \frac{u+1}{2}$$

$$t = \frac{v+1}{2}$$



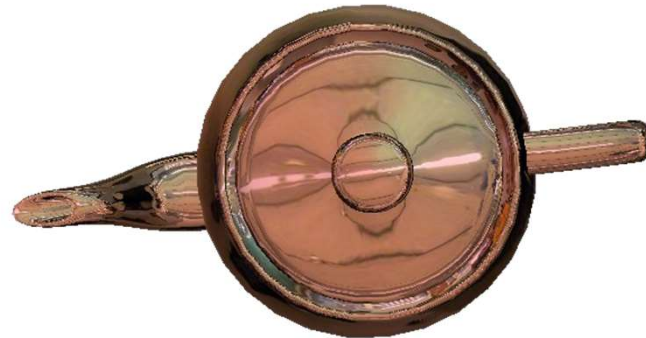
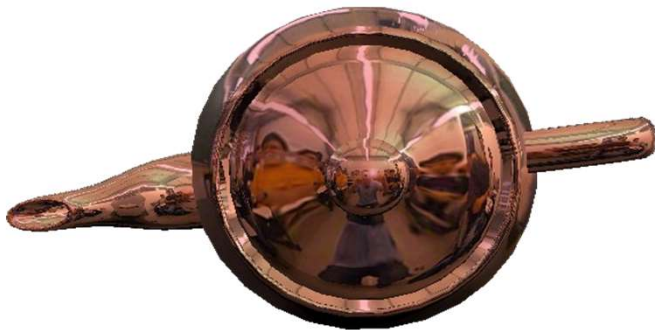
Càlcul del color donat R

```
vec4 sampleSphereMap(sampler2D sampler, vec3 R)
{
    float z = sqrt((R.z+1.0)/2.0);
    vec2 st=vec2((R.x/(2.0*z)+1.0)/2.0,(R.y/(2.0*z)+1.0)/2.0);
    return texture(sampler, st);
}
```

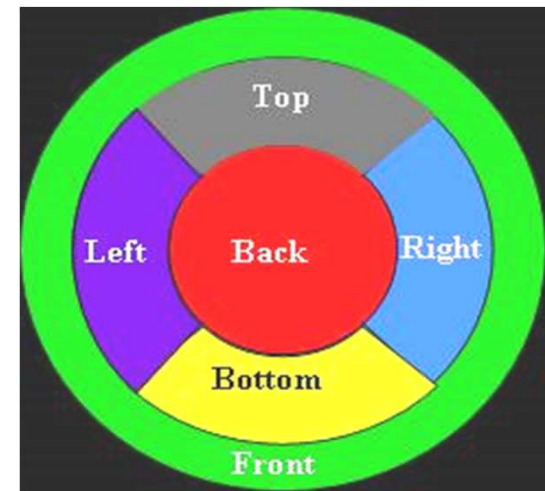
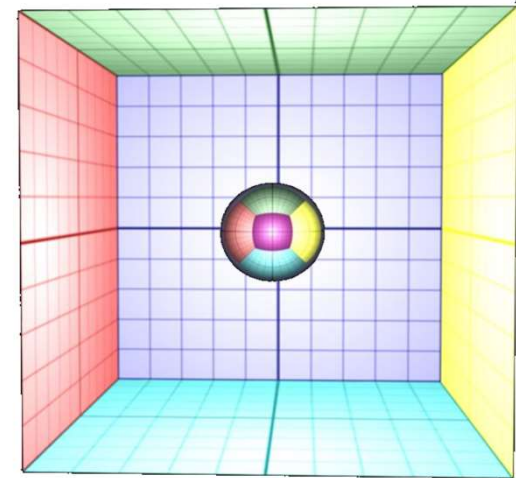
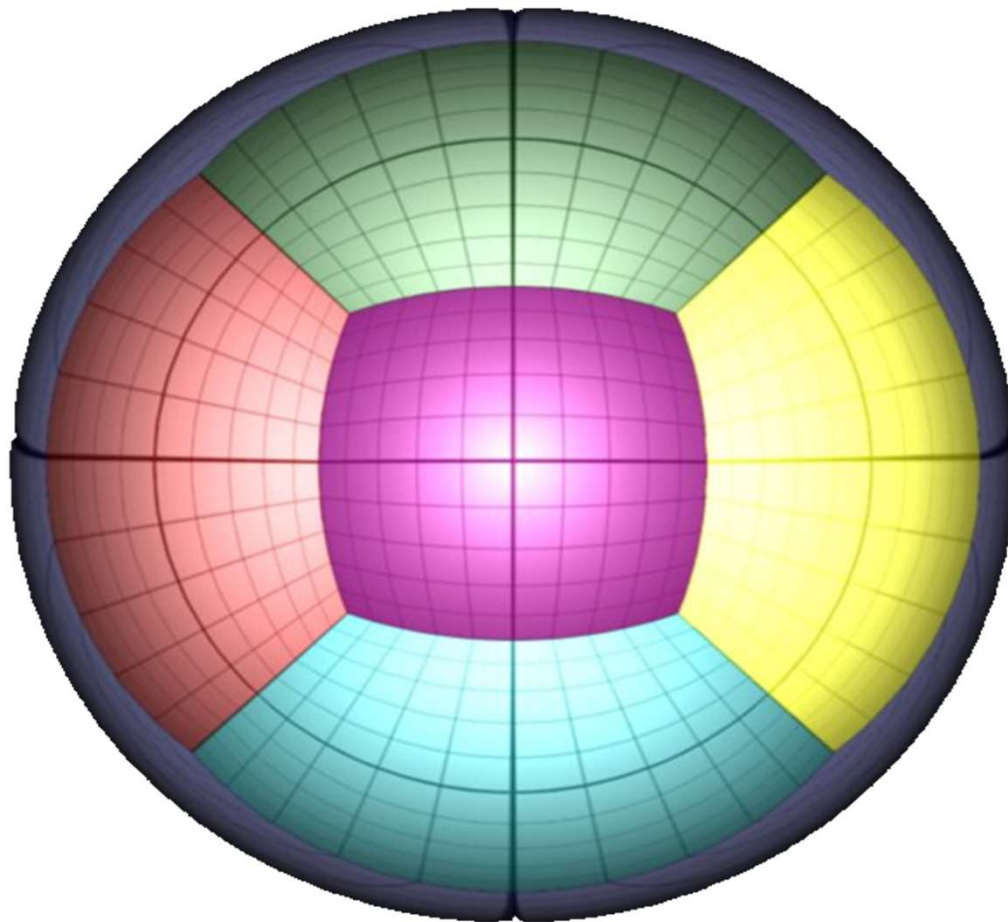
Eye/world coordinates



Eye/world coordinates



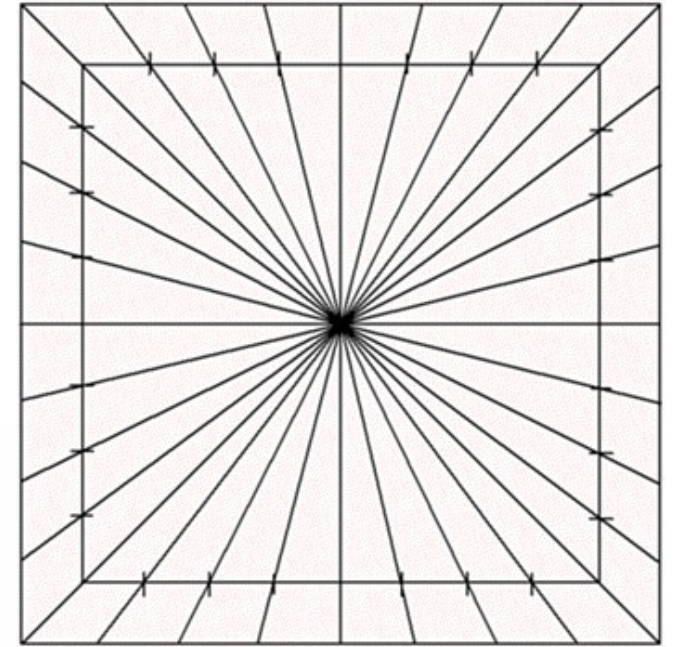
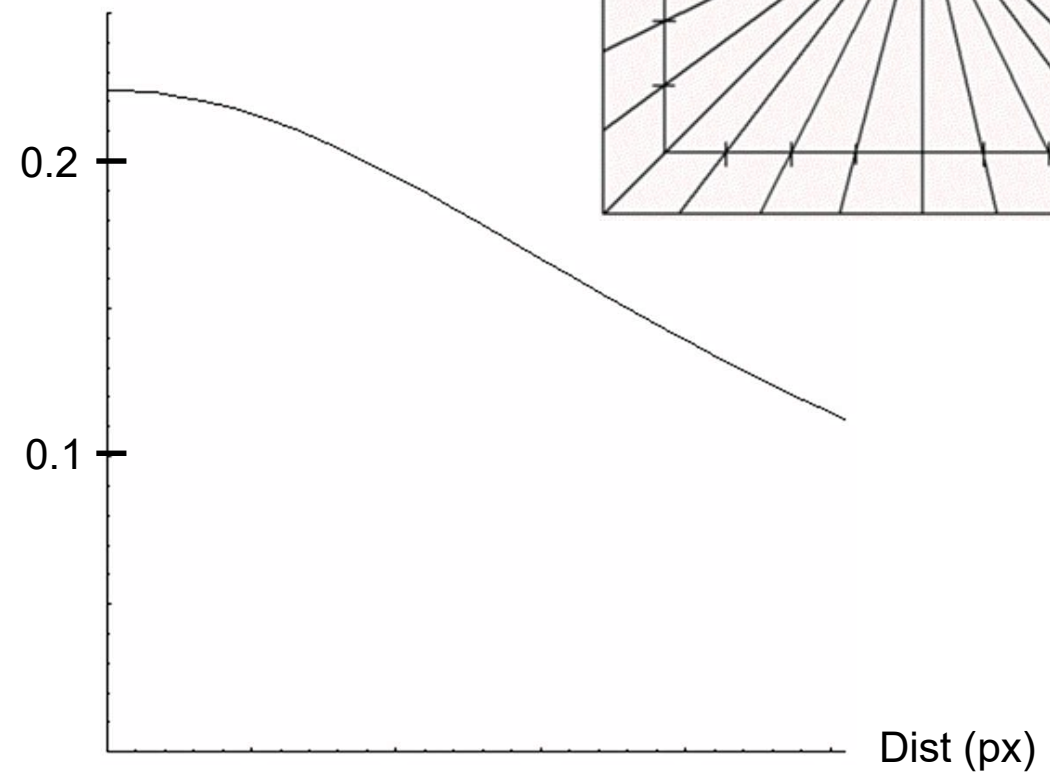
Sphere mapping: distorsió



CUBE MAPPING



Angle (deg)



Cube mapping: exemple



Cube mapping: exemple

// 1. Creació de les sis textures

```
glTexImage2D(GL_TEXTURE_CUBE_MAP_POSITIVE_X_EXT, ...);  
glTexImage2D(GL_TEXTURE_CUBE_MAP_NEGATIVE_X_EXT, ...);  
glTexImage2D(GL_TEXTURE_CUBE_MAP_POSITIVE_Y_EXT, ...);  
glTexImage2D(GL_TEXTURE_CUBE_MAP_NEGATIVE_Y_EXT, ...);  
glTexImage2D(GL_TEXTURE_CUBE_MAP_POSITIVE_Z_EXT, ...);  
glTexImage2D(GL_TEXTURE_CUBE_MAP_NEGATIVE_Z_EXT, ...);
```

Cube mapping: GLSL

```
uniform sampler2D sampler;  
...  
fragColor = texture(sampler, vtexCoord);
```

```
uniform samplerCube samplerC;  
...  
vec3 R;  
...  
fragColor = textureCube(samplerC, R);
```